DIGIC 7 findings from the EOS 6D Mark II port

A summary for Magic Lantern developers of what the 6D Mark II port (firmware 1.1.1) found, fixed and added that is not in upstream `magiclantern_simplified` (`dev` at `142472e95`, merged into this port's `main` on 6 November 2026). It covers fixes that matter to every camera, rules for DIGIC 6/7/8 work, capabilities new on this processor, hardware facts, and negative results that should save someone the time.

Everything here was tested on one camera over ten weeks unless it says otherwise. The full history is in the development log; the code is on branch `6d2-port` as one commit per feature on top of upstream `dev`. Lossless and the gyro log are on `main` only.

> **How this was made.** Nearly all of the code was written by an LLM (Anthropic's Claude), directed and tested on the camera by the repository owner. Under ML's `LLM_POLICY.md` that rules the code out as a contribution of substantial size. The **findings** — addresses, register and property meanings, measured behaviour, negative results — are facts, and are offered here so they can be used to write original code.

---

1. Fixes that matter beyond the 6D2

Bugs found in upstream code. Each was confirmed on the 6D2; the ones marked *all bodies* follow from the code alone.

Area	Problem	Fix
mlv_lite: second take	Teardown frees the buffers unconditionally, and reallocation waits for a state change a raw-only take never makes, so every second take was a 1 KB file. A take with the H.264 proxy masks it. *Possibly the root cause behind upstream issue 307* (the multi-record hang worked around with `PROP_LV_ACTION`).	Reallocation edge-triggered on the teardown; take start waits out the reallocation window.
mlv_lite: data loss *(all bodies)*	A take that stopped before creating its file ran `FIO_RemoveFile()` on `raw_movie_filename`, which still named the **previous** clip.	Cleanup removes only a file this take created (`created_file`).
mlv_lite: frame rate in the header *(all bodies)*	23976/1000 is 1 ppm off; some NLEs read it as non-standard.	NTSC rates written as 24000/1001, 30000/1001, 60000/1001.
mlv_lite: menu patched by index *(all bodies)*	`raw_rec_init()` hid entries by `children[N]`; inserting any entry retargeted those tweaks.	Look entries up by the variable they edit (`.priv`).
mlv_lite: any depth but 14	`raw_update_params()` fails below 14-bit on purpose (for overlays), and recording used it: "Raw detect error".	Recording uses an any-depth variant.
mlv_lite: proxy width bound	`use_h264_proxy()` compared against a hard-coded 2080; bodies with another default LiveView width silently never ran the proxy.	Per-body default width.

Area	Problem	Fix
mlv_lite: preview heuristic	`width > 2000` decides "framing incorrect"; a body whose ordinary LiveView is 2028 wide got ML's slow preview permanently.	Judged against the body's default width.
mlv_lite: aspect *(all bodies)*	When an aspect needs more rows than the readout has, upstream keeps the width and clamps the height, silently recording a different ratio.	"If too tall: shrink width / clamp height" option (default stays clamp on other bodies).
cropmarks: empty `ML/cropmks` *(all bodies)*	`crop_toggle()` does `MOD(x, 0)`; the Bitmap submenu reads `cropmark_names[-1]` (COERCE to -1).	Guarded.
Format, keep ML *(all bodies)*	`TmpMem_Init()` retried for ever with the UI locked when memory was short.	6 MB fallback, then give up after 5 tries.
mlv_play: index *(all bodies)*	A failed `.IDX` write left a 0-byte file.	Removed on failure; the loader rebuilds.
vectorscope (DIGIC 6+)	Chroma read without its +0x80 bias: every neutral pixel landed on the rim.	`UYVY_GET_U/V`.
`bmp_idle_copy()` (RGBA bodies)	Copied ~0.5 MB between Canon's RGBA layer and an address with bit 19 flipped — unrelated memory. `mlv_play` did it every tenth frame.	No-op on `FEATURE_VRAM_RGBA`.
raw preview (DIGIC 6+)	Hand-packed U = V = 0 is green on these bodies; neutral is 0x80.	`UYVY_PACK`.
10/12-bit playback	`raw_preview_fast_ex()` drew only 14-bit clips; the playback tone curve crushed the bottom stops.	Any-depth reader, 12 EV gamma 2.2 curve.

Also: the lua module did not link with current arm-none-eabi newlib.

---

2. Rules for DIGIC 6/7/8 work

- **Q inside a submenu closes it** (`menu_toggle_submenu`), on every body. A `.children` inside a child can't be reached, though it builds and draws normally. Check that a nested setting really opens.
- **Redraws skip a running take**, so an overlay turned off during one stays on screen until something clears its own pixels (`clrscr_mirror`).
- **Format card, keep ML** keeps `ML/FONTS`, `ML/MODULES`, `ML/SETTINGS` and files directly in `ML/`, not `ML/CROPMKS` or the rest. Don't let anything essential depend on those folders: `find_cropmarks` returned before marking cropmarks ready, which stopped every recorded-area guide too.
- `.advanced` **does not always hide an entry.** One whose value differs from its config default stays visible (`config_var_was_changed`), so an entry with a "not set" default resolved at init needs its `default_value` updated. Learned on the 6D2's RGBA compositor; probably true of every `FEATURE_VRAM_RGBA` body.

1. **Canon never clears ML's layer.** On DIGIC 4/5 Canon repaints its bitmap and wipes ML's graphics; here ML's shadow buffer persists. Every LiveView mode change, menu close, playback return or quick review needs an explicit `redraw()` / clear — **but never a full clear while recording** (it produced a Canon ERR on the next record press, reproducibly).

2. **Anything that draws in LiveView must be gated on `liveview_display_idle()`, never on `lv`.** Each draw that sets `ml_refresh_display_needed` makes the compositor push ML's layer over whatever Canon shows. A meter task drawing at 20 Hz gated on `lv` broke every overlay and Canon's own UI.
3. **`redraw()` on DIGIC 6/7/8 repaints `LiveViewApp_dialog`.** If Canon has another dialog up (the format dialog), that repaint hides it and its buttons until the user turns a dial. Repaint the open dialog instead.
4. **During a take, `redraw()` clears nothing**, so every `NotifyBox` shown mid-take used to stay up until the take ended. Small fills of a known rectangle are safe mid-take.
5. **Spaces draw nothing on the RGBA layer** — padding a string with spaces does not erase the previous one. Fill the rectangle (colour 0 is transparent).
6. **Chroma is biased by 0x80.** Use `UYVY_PACK` / `UYVY_GET_U/V`.
7. **`call()` passes a pointer to the vararg array**, not your struct; eventprocs that read `ldrb r0,[r0]` want values *in* that array.
8. **Canon's `vsnprintf` has no `%c`** (it pads instead — a `%c` path made a directory of 23 B's). ML's own printf mishandles field widths of 10 and up.
9. **Canon drops a LiveView zoom request made while it is still settling**, and `set_lv_zoom()` returns on Canon's first broadcast, before the view arrives. Wait for the readout (DMAC width), not the property, and re-send.
10. **Never key a visible retry on the state it changes** (a retry per *(depth, zoom)* fed itself through every magnify press).
11. **Platform switches can be silently absent.** `CONFIG_MOVIE` undefined and one denied property kept the H.264 proxy dead for seven weeks; check what a feature compiles to first.
12. **No static buffers in the core.** The 6D2's `user_mem` has ~11 KB of headroom; a 2.5 KB event log in `gui.c` broke LiveView overlays. Diagnostics belong in a module.
13. **The platform build reuses stale modules** — `make clean` in `modules/` after a header change.

---

3. New on DIGIC 7 (not in upstream)

LiveView overlays actually showing

Zebras, peaking, false colour and the graphs never appeared on any RGBA body: they drew through `bmp_vram_real()` (Canon's 32-bit layer, not ML's shadow buffer), and only `menu.c` / `rbf_font.c` set `ml_refresh_display_needed`, so the overlay pass was composed and discarded. With `bmp_vram_overlay()` and the flag set from the overlay passes, the whole set works, including magic zoom and flip-screen support.

6D2 LiveView geometry (from the DISP struct at `*(0x7dc8)`, size at `+0x94/+0x98`): the buffer is **736 wide**; the picture is columns **0–719** and 720–735 are a flat padding band (measured from per-column luma). Photo LiveView is 736×480; movie LiveView is **736×408**, letterboxed at rows 36–444 of the 480-row panel. Upstream's hard-coded 480 read 72 rows past the buffer.

Flip screen (`PROP_LCD_POSITION`): position 1 (facing forward) mirrors Canon's image layer left-right but not ML's; position 2 (flipped outside) needs nothing. Reflecting the `BM2LV_X` table moves every overlay at once, but *distances and anchors* must use the unmirrored mapping.

10/12-bit LiveView raw

The first bit-depth switch on a DIGIC 7 body, through Canon's own raw tap rather than `EngDrvOut` (an empty stub here). The Mem1 raw-path update at **`0xe01470fe`** hard-codes the depth twice: bits per pixel (`mov.w r0,#0xe`) and a PACK32 code (`mov.w r0,#2`; 0 = 10, 1 = 12, 2 = 14). Patch both immediates through the MMU patch manager and bounce LiveView (zoom out and back) for Canon to rebuild the path. **Read the depth back from the write DMAC's pitch** — the readouts are 2128 or 3264 wide, so exactly one of 14/12/10 divides out — and a stale or failed switch can never mislabel data. Arm while the menu is open; apply once when it closes; hold the depth between takes (a bounce at record stop ran into Canon's teardown).

Video dual ISO in the 1:1 readout

The LiveView ISO tables are in the sensor-config block (ROM `0xe1980000`, DMA-copied to the heap; find the copy as `dual_iso` does): four 26-row tables at `+0x0ef4 / +0x11cc / +0x14a4 / +0x177c`, 1/3-stop rows of 7 CMOS words (stride `0x1c`). The per-frame sensor update (`0xe0525410`) copies row `LUT[iso]` from **ST2 (`+0x14a4`) in normal LiveView** and **ST3 (`+0x177c`) in the 5×/10× zoom modes** into the list `CMOS_write` (`0xe053cdc4`) sends every frame, so a patched row is live one frame later with no ISO nudge. (The 200D notes in `dual_iso.c` already observed a table that changes x5/x10 zoom.)

- **Zoom (1:1) readout:** works — low-low-HIGH-HIGH line pairs, cr2hdr / MLV App compatible, at 14, 12 and 10-bit.
- **Binned 1080p:** the patched ST2 row demonstrably reaches the sensor, yet every line pair gets the same gain. The binned readout applies one gain; leave dual ISO inactive and the MLV untagged.
- **Gain is partial:** ISO 100 / recovery 800 differ by **3.3× (1.7 EV)**, not 8×, on unclipped pixels; base 1600 / recovery 800 gives the full 1 EV. The per-line field covers only part of Canon's ISO gain, so ML's "DR gained" estimate is optimistic on this body.

On lossless crop takes (v130 and v131), the line pairs hold on every frame: bright rows over dark 3.18 to 3.32, or 1.7 EV, at ISO 100 with recovery as set, and the gain is flat with brightness (3.25 to 3.34 from 16 DN to about 1000 DN over black, per colour). Which rows are bright depends on where the recording window starts, so measure the phase per clip. The `DISO` block's `isoValue` read 3 on these and did not give the gap.

Crop mode from Canon's own zoom

There are no readout-window registers ML can drive here, but Canon's 5×/10× LiveView magnification **is** a 1:1 sensor window (3264×942 raw, **3164×896 active**: rows 44 to 939). Holding `PROP_LV_DISPSIZE` = 5 while RAW video is on gives a stable crop mode up to 3072 wide. It needs `PROP_LV_DISPSIZE` in the write whitelist, and it is **always 29.97 fps**, whatever the movie mode. Lens AF (any AF method) makes Canon refuse the zoom.

H.264 proxy on DIGIC 7

RAW and Canon's H.264 on one card, paired by number (`IMG_1234.MLV` ↔ `MVI_1234.MP4`); 12.5 minutes at 2048×856 12-bit plus a complete MP4 confirmed. Three faults, each hidden behind the previous:

1. Reallocating while Canon was only *starting* took the memory Canon's recorder needed (no MP4); wait for `RECORDING_H264_STARTED`, with a bounded wait (the old wait needed `gui_state` idle, which a recording camera never reports).
2. `CONFIG_MOVIE` **was never defined for the 6D2**, so `movie_start()` / `movie_end()` were empty.
3. `PROP_MVR_REC_START` **was on the 6D2's do-not-read list**, so `RECORDING_H264` stayed 0. Of the handlers it wakes, only `audio-common.c`'s `enable_recording()` touches hardware (an AK codec task this body lacks); skip that one and read the property.

Scratch audio without ASIF

DIGIC 7 exposes no ASIF stubs, so `mlv_snd` cannot run. Canon's factory eventprocs include a complete WAV recorder: `FA_AudioRecStart` / `FA_AudioRecStop` (`0xe008c0a0` / `0xe008c0ac`), 48 kHz 16-bit stereo, ~0.19 MB/s, written as `B:/Sample%d.wav` (path built at `0xe064fc6e`). Facts that cost time:

- `FA_AudioRecStart` takes the take number **by value** in the vararg array.
- `FA_AudioRecStop` on a recorder that is not running asserts (`SoundDevice.c:469`); renaming before Canon closes the file asserts (`FileIO.c:119`) — wait for the size to settle.
- The recorder **appends** to an existing `SampleN.wav` (a 25 s take carried 414 s of audio); advance past numbers already on the card.
- The mic is chosen by a **factory flag, not jack detection** — set it with `Test_AudSetMicAmp` (0 internal, 1 external) before starting.
- Input gain: Canon looks the level up in a 64-entry table per input and writes it to `0xd8000550` **(L) /** `0xd8000554` **(R)** in 0.375 dB steps — and writes the **left** value to **both**, so a different right level is ML's to write. The factory default is quiet (~−41 dBFS).
- `0xd8000190` (what Canon's meter reads, `0xe04b576e`) holds an average **level** and a hardware-held **peak**, not left and right — drawn raw it looks like a level bar and a peak bar, while the WAV's two channels differ by ~6 dB.

Format card, keep ML on DIGIC 6/7/8

Canon's format screens are WinSys dialogs that are not on `gui_task_list`, and their string slots differ from DIGIC 4/5 (and `MENU_MODE` does not hold), so the DIGIC 4/5 hijack cannot work. Each format module keeps its globals in a small block with the dialog handle at `+4` (`StopMnCardFormatBeginApp` `0xe0718044` deletes `[0x229cc+4]`; `StopMnCardFormatexcuteApp` `0xe074f25a` deletes `[0x27730+4]`), so **`DIALOG_MnCardFormatBegin = 0x229d0`** and **`DIALOG_MnCardFormatExecute = 0x27734`** on 1.1.1. The watcher keys on the Begin handle, tells the user through NotifyBox, waits for the Execute dialog to appear and clear before restoring, rewrites the boot flag with the installer's own function (exFAT confirmed), and asks for a power cycle (`PROP_REBOOT` is not writable). The R sets `CONFIG_RESTORE_AFTER_FORMAT` with the DIGIC 4/5 path.

Top-plate buttons, down to the MPU

AF, ISO, DRIVE, metering, the AF-point button and the light button have **no button codes** on this body. Each press is the generic `GMT_GUICMD_PRESS_BUTTON_SOMETHING` (`0x4b` here, `0x54` on the 550D), then Canon's reaction as `GMT_OLC_INFO_CHANGED` (`0x62`): AF arg `0x16` / `0x11`, ISO `0x1e`, DRIVE / metering / AF-point arg `9` with obj bit 26.

One layer down: Canon's SIO3 interrupt keeps the last 80 received MPU messages in a ring of pointers (**`0x80d84`**, index **`0x7c00`**, message at pointer + 4, first byte its length); it can be polled read-only. On 843 messages: every press is MPU `06 05 06 26 01 00` (switch 0x26 — the generic event, consistent with the "user did something" reading), then the MPU's decision as `0a 08 <id> …` (the `0x62` event; its `obj` words are those bytes). In movie mode DRIVE, metering and AF-point all send `0a 08 20 00 00 04` (show "not available"), byte for byte; in photo mode they open different dialogs and AF-point's flag is `08`. **The button identity never leaves the MPU** — they cannot be told apart. The light button and the rear dial both send the generic message alone.

Temperature

The 6D2 MPU never sends `0x80030014` (ML's `PROP_EFIC_TEMP`). It sends the **board temperature** `0x80030035` (MPU `03 38`) about every 30 s; **raw − 128 = °C** (a camera left off in a 21 °C room read 22–23 °C at power-on). Canon's own warning and shutdown follow `PROP_TEMP_STATUS` (`0x8004000F`; MPU `01 3d` in QEMU's tables for other bodies, not yet seen in a 6D2 log). (The R's `test_features.c` names `0x80030035` `PROP_R_TEMP_MAIN`.)

Writing the shutter on DIGIC 7

`PROP_SHUTTER` (`0x80000005`, 4 bytes) accepts ML's requests in movie Manual exposure exactly like the dial, once whitelisted — the basis of a shutter-angle lock (nearest 1/3-stop speed by Canon's nominal value: 180° = 1/50 at 23.976, 1/60 at 29.97).

MMU patch pages from the heap

The stock MMU patch pool backs one remapped 64 kB page; with `user_mem` at its limit a second ROM page was refused (`BAD_MMU_PAGE`). `CONFIG_MMU_EXTRA_PAGES` backs further pages from the heap. (Review note: the allocation runs with interrupts off and the second core parked — pre-allocating before `cli()` would be safer.)

Lossless raw video on Canon's CR2 encoder

mlv_lite records **14 and 12-bit lossless MLV (LJ92)** on the 6D2 1.1.1 at 23.976 and 59.94p, using the encoder Canon uses for CR2 photos (`LosslessEncode.c` on JpCore). The backend is a 6D2 branch of `modules/silent/lossless.c` behind `lossless_compress_raw_rectangle()`, so mlv_lite's existing DIGIC 5 lossless pipeline (double-buffered full-size frames, compress task, slot shrinking) runs unchanged on top of it. Verified on the camera:

- **Bit-exact.** Encodes of grabbed LiveView frames decode to exactly the source pixels at 14 and 12-bit (d7probe's encode test against `lj_verify.py`), including with the final routing setup below. This holds in the binned 1080p readout (1920×800 crops) and in the 1:1 crop readout (3072×896 out of 3264×942, 14.2 ms per encode against crop mode's 33 ms frame).
- **Every recorded frame decodes.** A strict decoder passed all 5,520 frames of nine test clips at 14 and 12-bit, 23.976 and 59.94p. Each frame must use exactly its declared rows and end within the last byte before EOI. MLV Drive serves the frames as uncompressed CinemaDNG, pixel-identical to an independent decoder.
- **Noise identical to uncompressed footage** of the same scene: single-frame flat-block noise, lossless / uncompressed, median 0.996 (14-bit) and 1.002 (12-bit) across the signal range.

The recipe (Thumb addresses, 1.1.1), in Canon's order with ML's record swapped in:

Step	Address	Notes
open	`0xe015936a`	resources `{0, 0, share-mem table, count, 0, 0x30, 0x11, 0x03}`: read / raw-write / thumbnail channels. **Pass count 0** (below)
record	`MEM(0x26584 + 4)`	swap in ML's: width, height, pitch in pixels, crop x/y, depth code (2 = 14-bit, 1 = 12-bit), thumbnail area; restore Canon's after close
callback	`0xe0159348` / `0xe0159358`	set / unset; `cbr(ctx, a, b, c)`, nonzero `b` / `c` = refused
DEGEEN	`0xe01594d6`	32 words; **identity** = offset 0, words 4–7 = 1, gain `0x400`, add-back 0, clip `0xffff`. The photo's block shifts every pixel by one; a zero clip zeroes the frame
TWAIN	`0xe01594de`	the thumbnail's; Canon sets its black words to black + 1
Huffman	`0xe02474ec`	`(ram_table, spec)`; spec = 2 × (16 counts, 16 values, 16 zeros)
parameters	`0xe015945c`	`{table, 1, memory list, input address, thumbnail buffer}`; list = `{output, size}`
start / stop / close	`0xe01594ce` / `0xe01594d2` / `0xe0159422`	

- JpCore takes **12 or 14-bit only**, and reads the crop straight from the full-size packed frame. Under 4096 lines it writes **2 components of width/2 × height** with predictor 1 and point transform 0: MLV's LJ92 layout, so MLV App and MLV Drive decode it as is.
- **Tables:** Canon refuses more than bpp + 1 symbols (categories 0..bpp) and codes longer than 10 bits: a 16-symbol table came back with callback `b = 1, c = 0x041c001c`. ML's video tables (in `lossless.c`) give 41–46% at 14-bit and about 33% at 12-bit, against Canon's still table at 50%.

- **Done:** in LiveView the callback is often missing. The compressed size appears at base state `0x2b084 + 0x18` (set it to -1 before the start), and the EOI landing at `output[size-2..size-1]` marks the output DMA done.
- **Wait on an event, not a sleep loop.** Polling with `msleep(1)` and the millisecond clock made a 9.5 ms encode cost 29 ms at 59.94p (no take lasted past 69 frames). A semaphore given by the callback or by a 250 µs `SetHPTimer` poller of the size and EOI gives 9.3–11 ms. The poller counts its own ticks (no clock reads in interrupt context) and carries the encode's number.
- **The thumbnail unit always runs.** It needs its own buffer (256 KB with a guard word; ~30 KB used at 1920×800).

LiveView's green, and the fix. Open applies the share-memory table from its resources (Canon's at ROM `0xe0942290`, 4 elements) to `0xd0003200/04/38/3c`, re-routing LiveView's memory to the encoder; close clears it. LiveView's values are `0x80000000 / 0x80010000 / 0x80060004 / 0x80070004`. With them gone, Canon's preview (and the H.264 proxy) loses its colour until LiveView restarts. Every step of the way out was measured:

1. Canon's unit setters write each value twice: the register, and a RAM image at `MEM(0x1faa4) +` offset from `0xd0000000` (`0x021d0800`). **Restoring the RAM image alone does nothing.**
2. Restoring every word the encode changed, registers included, kept the colour but crashed the camera within a second (`LvJob.c:203`, `EvfState.c:660`). Most of those words are LiveView's per-frame buffer addresses and toggling flags, and they were stale.
3. Restoring only the four routing words after close fixed single encodes. Encoding every frame, though, LiveView stayed green (half the screen at 59.94p).
4. Restoring them right after open left only a flash at the top of the picture, from inside open.
5. **The encoder does not need the routing.** Apply (`0xe0247e30`) and clear (`0xe02480b8`) return at once when the element count is 0, leaving only `(SetTable)ElementNum is Zero` in a debug pointer at `0x1f074`. With count 0 LiveView is never touched: the preview is clean, framing, peaking and overlays work, and the encode stays bit-exact at the same speed.

Memory. While RAW video is on in movie mode, mlv_lite holds the whole shoot pool and the SRM buffer. `shoot_malloc_suite(0)` with nothing free then asks Canon for 0 bytes, which resets the camera. Ask for exact sizes.

A gyro in the body

Canon lists no gyro for the 6D2, but its electronic IS driver (`./ElectronicIS/Driver/DSLR/EIS.c`) keeps three-axis gyro data in plain RAM, and ML can read it without calling anything:

Address	Contents
`0xd9008664` / `68` / `6c`	pitch, yaw, roll, clamped to +-24960
`0xd9008610` / `14` / `18`	the same before the clamp, about 16x the scale, new about every 249 us (4 kHz)

`GetGyroData*` and `GetGyroHPFIN*` only load these words. Seven d7probe runs (`Gyro test`), five on a lens reporting no name and two on an EF 24-105 f/4L IS II: each axis answers its own motion on both lenses, with a steady per-axis offset at rest (yaw about -300, roll about -90, pitch about -10). The 4 kHz rate rules out image motion vectors as the source. Fast deliberate rolls hit the clamp; the filter-input words reach only about 5% further (+-420000 at most, about 26000 on the same scale).

It keeps working during a lossless RAW take. M04-0845 (2048x1152 12-bit, 1296 frames, 54.0 s, none lost) ran with the test polling at 100 Hz and a 300 ms busy read inside the take. Frame-to-frame image shift against the integrated gyro gives one offset from yaw and from pitch independently (gyro row 0 at clip time 6.35 s), which also shows the log and the picture are the same motion.

Scale, yaw only, from that take on a 55 mm lens: about **80 LSB per deg/s**, so the clamp is about 310 deg/s (87% of the 84 fastest usable frames within 20% of the ratio; 3x binned pixels of 17.25 um assumed). Pitch and roll did not give a clean figure from the image, so assuming the same scale on them is untested. On a countertop the noise is 10 LSB (about 0.12 deg/s). Yaw's offset moved from -242 to -290 across three runs in a few minutes, so it needs measuring per take.

The filter-input words already have the resting offset removed: they average about zero at rest, where yaw's clamped word sits near -290.

The log (v122, marked experimental in the menu): `Gyro log` in mlv_lite samples all six words at 1 kHz from an HP timer, stamped on the VIDF clock, and writes them **inside the clip**: one `GYRI` block, then `GYRO` blocks between frame writes (24 KB/s). mlv_lite's block queue is the wrong path for this, since it is only written at chunk start and take end and holds 100 blocks. `mlv_dump` and MLV Drive step over the new blocks. `tools/gyro_export.py` writes Gyroflow's `.gcsv` from them.

`SetHPTimerNextTick` (`0xe06261a2`) calls the **overrun callback from inside the call** when the next tick has already passed, so a timer that is its own overrun callback nests on the interrupt stack while interrupts are held off. **Never give an HP timer its own function as the overrun callback.** v123 fixed this in the gyro log and v124 in the encoder's poll timer. It was not the whole story: resets at the first frame carried on with the gyro log off (below).

The gyro-on hang was a loop, not the card. `gyro_log_write` wrote until the ring was empty while the timer kept adding samples, so it never returned to the frames: `M04-1409` holds 23 frames and 11007 GYRO blocks, written for 43 s until the power went off. The other tasks filled the buffers, hence the counter frozen in red at about 150 frames, and the task that would end the take was the one stuck. v125 writes what was there when called. An earlier guess blamed the file offset; v124's 512-byte padding did not stop the hang, and stays only because frames and headers keep that alignment for write speed.

A boot's first take at 59.94p reset the camera at the first frame, 5 of 5 on 2026-11-04 (v122 and v123, 3 with the gyro log on and 2 with it off, all WAV off, shutter 1/1000), with no crash log and a 1 KB clip. The same boot started with a 23.976p take was fine at 59.94p afterwards. First takes at 59.94p with WAV on and 1/125 had passed on 11-02 on earlier builds, so WAV, shutter, the builds since, and the card format are all still open. The only first-take difference found in the code is the encoder's 256 K buffer, allocated at the first frame until v124.

More takes on v123 the same day made it 9 of 9, all gyro off in the last 4, and two left crash logs: `LvJob.c:203`, `g_LvJob[ m_dwLastJobNumber ].fEnableInstance`, task Evf, free memory already 256 K down, so the encoder's buffer had been allocated. The assert's string sits beside `LiveView CreateJob BufferOver` in the ROM (`0xe0072c88`; string order only, not disassembled), which suggests LiveView's job ring filling up because jobs stopped completing. A ring would fill 2.5x sooner at 59.94p than at 23.976p, which fits the pattern. Why jobs stall is not known.

On v124, with the encoder's buffer allocated before recording and its poll timer no longer nesting, 3 of 3 boots passed the first 59.94p frames and first write, one of them a full take. Which change did it, and whether it holds, is still open: 3 boots is little against 9 of 9 failures.

v125 ran the log for real on 2026-11-04: `M04-1420` and `-1421`, 59.94p, 4146 frames with none dropped, 69748 samples with none lost, 1 overrun per take, longest gap 2 ms, and encode time no higher than with the log off. With v124 and v125, 6 of 6 boots got past a first 59.94p take. Two takes started 3.6 s and 6.3 s after boot ended by themselves with no frames and no clip.

Those moves were too fast to calibrate well (median correlation peak 0.32 to 0.43). Yaw and pitch come out at about 80 to 90 LSB per deg/s, assuming the Takumar 55, against 80 from the slow pan; roll is unresolved, and strip readout ranged from -67 to +18 ms. Slow sweeps are needed. RGB keyboard flicker (`M04-1426`, about 326 Hz) gave a line time of 20.8 to 20.9 us against 21.17 us from the timers, but its phase is not straight down the rows (band period 125 at the top, 151 at the bottom), so it supports about 21 us, not an exact figure. Mains light is the better test.

Readout, measured. The binned 1080p readout takes **10.59 us per row**, at both 23.976p and 59.94p, half the 21.17 us timer B tick: two rows per tick. Mains flicker on a plain wall under a dimmable light (`M05-1725`, `M05-1727`, ISO 1600 and 3200, only about 50 DN over black) gives bands 786.5 rows apart at 120 Hz, phase rms 0.002 cycle. The 240 Hz and 360 Hz harmonics agree (393.7 and 262.2 rows), the bands drift at the 120 Hz alias rate at both frame rates, and there is almost no 60 Hz. Separately, top and bottom strips against the gyro in `M05-1734` (804 rows) give 8.3 +- 0.9 ms, against 8.5 ms predicted and 17 ms for a whole tick per row. That is 7.63 ms for 720 rows, 9.07 for 856, 12.2 for 1152 and 12.8 for the full 1208. The earlier 15 ms was wrong, and the keyboard flicker only seemed to support it because that test assumed 600 to 1000 lines per frame. The crop readout is unmeasured.

Scale from the same round's fast frames: pitch 83 to 89 LSB per deg/s across five clips, yaw a looser 62 to 86; roll still unresolved. The takes were dark, median correlation peak 0.26 to 0.47.

Canon's digital IS changes nothing measurable. `M05-1739` (on) against `M05-1740` (off), same 24-105 at 24 mm, 1600x670 59.94p: word to filter-input gain 1.00 on yaw and roll in both (pitch 1.00 and 1.03), same offsets, no clamping. The raw picture follows the gyro at least as closely with it on (6 to 30 px per frame: pitch 0.98 of prediction on, 0.58 off; yaw 0.67 on, 0.51 off), where stabilised raw would move less. The off take tracked worse (median correlation peak 0.26 against 0.33), so these are not exact ratios, and the moves were not identical. See gyro-test.md.

Where the gyro stands (v126).

Reading it	Works, including during lossless takes at 59.94p
Log in the clip	Works: 4146 frames and 69748 samples with none lost
Readout	Measured for binned 1080p; crop readout unmeasured
Scale	Pitch 83-89 LSB per deg/s, yaw 62-86, roll unresolved; `gyro_export.py` assumes 80 on all three
Signs	From image motion on pitch and yaw; roll's direction is not confirmed
Gyroflow	Logs are written in its format, but not yet tried in it

Next: spins on a table for the scale on each axis, one bright take for roll's direction and delay, then a Gyroflow trial.

Write speed against card fill (SanDisk 64 GB, exFAT, formatted before the first of these takes). Sustained 59.94p takes, large writes: about 74 MB/s empty, falling 0.16 +- 0.05 MB/s per 1% of the card used (12 takes, 8% to 72% full, rms 1.8): about 70 at 25%, 66 at 50%, 62 at 75%. Fill and board temperature rose together (correlation 0.80), but 65.6 MB/s at 48% and 35 C points to fill. Short dips are the bigger change: the first 20 MB of a take wrote at a

median of 73 to 75 MB/s under 50% full and 54 MB/s (35 to 76) above it, with one 17 MB/s stall. Takes at 23.976p and 29.97p held 72 to 76 MB/s to 52% full. Before the earlier format the same card gave 49 to 67 MB/s at 72% to 81%. Noise matters as much: 1920x804 at ISO 6400 needs about 66 MB/s, against 42 to 45 in a clean scene.

59.94p above 1920x960 fails. 2048x1152 at 59.94p encodes in 16.3 ms on average, 18.4 at most, against a 16.68 ms frame, with 47 of 68 encodes missing the callback. `M05-1726` (2048x1152) reset at the first frame with no crash log. `M05-1732` (2048x1024) took 1233 ms over its first write, with 8 gyro overruns, then asserted at `EvfState.c:660` and `:584` (`TryPostEvent` failing, so Evf's queue is likely full; not disassembled).

A tighter ratio is what makes full width work at 59.94p. Aspect ratio only chooses which rows are stored: the sensor still reads its whole window on every frame, so readout and frame timing do not change. Fewer rows do lower the data, the encode time and the skew. `M06-1254`, 2048x856 at 59.94p (1.75 MP), recorded 2563 frames. 1920x960 (1.84 MP) has worked and 2048x1024 (2.10 MP) has not. In crop mode rows are the limit, so a tighter ratio allows more width: 2136x894 at 2.39:1, 3072x614 at 5:1 (encode 11.1 ms of a 33.4 ms frame).

Why the audio isn't inside the clip: MLV has blocks for it (`WAVI`, `AUDF`), but `mlv_snd` needs ASIF, which DIGIC 7 doesn't expose, so the 6D2's scratch audio comes from Canon's own WAV recorder.

4. 6D2 hardware facts and measurements

Frame timing. `TG_FREQ_BASE` is **44.636 MHz** on this body (upstream's 66.8 MHz tagged 23.976 footage as 35.9 fps), solved from VIDF timestamps. Canon's timer getters assert (`ImgSeqCoopStore.c:194`) for a moment during every LiveView mode change — read the store (`0x1c928 + 0x1c` for timer B) directly. The binned readouts are paced by timer B (estimated ceiling ~36–37 fps for the full 1208-line 1080p readout; the 59.94p readout is a full 2128×1208 too, and is sensor-paced - its timer values do not change its rate). The 1:1 readout is paced on the sensor side.

Black level. The 1080p raw is binned 3× across, so only ~40 columns are optical black (ML measured 16–84, from the full-resolution CR2 border). Headers read 2128–3654 against a true 2047. Pinned at **2047 / 512 / 128** for 14 / 12 / 10-bit.

Readouts. 1080p: 2128 wide with **2080 active columns** (2048-wide recording is clean to all four edges). 1080p's picture starts at row 28 and runs to the last row, so 1180 rows. Zoom: 3264×942, 3164×896 active. Judge the readout from the DMAC width, not `PROP_LV_DISPSIZE`, which flips before Canon rebuilds.

ISO in video (lens-cap dark frames, 14-bit):

ISO	read noise	DR
100	6.23 DN	11.15 EV
400	7.40	10.90
800	7.60	10.86
1600	9.26	10.58
3200	11.80	10.23
6400 (3200 + 1 EV digital)	20.76	9.41

Analog gain stops at 3200 in video. Strongly ISO-variant: up to 3200 each stop costs ~0.14 EV while input-referred noise improves 17×. Third stops are a full stop plus a digital push baked into the raw.

Bit depth. 12-bit is lossless in practice; 10-bit costs 0.42 EV of shadow range at ISO 100 (measured; the step/√12 model says 0.32). Canon's packer has codes for 10/12/14 only.

Lossless. At 2048×856, 14-bit frames are 1.33 MB (43% of uncompressed) and 12-bit frames 0.84–0.90 MB (33%); both vary with the scene. Encode time is 8.2–9.9 ms from start to done, plus about 1.1 ms of setup and stop: 9.3–11 ms in all, under the 16.7 ms frame period. Data rates for 12-bit 59.94p:

Width	Data rate
2048×856	~49 MB/s
1920×804	~42–45 MB/s

The **59.94p readout is a full 2128×1208** (the same as 23.976), so 60p loses no lines. Scratch audio works at 59.94p: 48 kHz in real time, with the usual ~0.38 s lead-in.

Card writes are bound by write size. Writes per take, 59.94p 12-bit lossless 1920×804, on a card that benchmarks 70+ MB/s:

Frames per write	Average write	Speed
2–3	1.9 MB	18 MB/s
4–7	4.1 MB	40 MB/s
8–15	7.3 MB	49 MB/s
16–31	14.9 MB	56 MB/s

mlv_lite's writer drained whatever was queued. Lossless frames finish one at a time, so it mostly wrote 4 MB at 40 MB/s. A 3 min 11 s take survived at ~42 MB/s with the writer busy 99.7% of the time; busier scenes stopped.

Holding lossless groups to ~20 MB (v116) raised 59.94p writes to **59–63 MB/s**, mostly 8–31 frames each, and a 2048×856 take ran without stopping. The hold must count *usable* slots. Each slot group carries as many reserved slots, which only become usable as compressed frames shrink, so a test against the total held nothing at 23.976 (v116 wrote single frames there; fixed in v117).

Cards. The SDR104 path reads ~100 MB/s and Canon already runs its fastest divider; the limit is the card's write speed. A SanDisk Extreme PRO 64 GB writes ~83 MB/s new, **~62–65 MB/s after one full fill** (a camera format does not restore it). Sustained recording needs the card's *slowest* writing, not its benchmark average. macOS AppleDouble `._*` files collide in FAT 8.3 short-name space and make clips look corrupt.

Audio. The internal mic delivers two genuinely different channels (right ~6 dB under left, correlation 0.16 on a clap take).

Addresses (1.1.1) — beyond the ones above: `call 0xe04e7432`; photo CMOS ISO table `0xe1980b30`; ROM1 tune-data store at ROM1 offset `0xb30000`; timer-B apply `0xe03029bc`; LV prepro update `0xe014493e`; `SetSDClkFrequency 0xe0292d1c`; `sd_device 0x20054`.

---

5. Negative results

Tried and ruled out on the 6D2 — recorded so nobody repeats them blind.

- **24p in the 1:1 readout**: five ways (timer B, Canon's timing lists, ROM list patches). Canon re-asserts the frame period on the sensor side; always 29.97.
- **48p from the 59.94p readout.** Timer B × 1.25 through Canon's ImgSeqCoop store (`0x1c928`) and apply: B snapped back, timer A moved (368 → 472), and the frame timestamps still measured 59.6–59.7 fps. Like the 1:1 readout, the 59.94p one is paced on the sensor side.
- **Lossless: delaying the encode into the frame period** (2.5 / 5 ms after vsync), to dodge LiveView's processing: the preview went fully green and the callbacks stopped. At 5 ms the encode overran the frame. The empty share-memory table (section 3) is the fix.
- **Lossless: restoring Canon's RAM register image** (`MEM(0x1faa4)`) has no effect on LiveView; restoring LiveView's per-frame registers crashes it (section 3).
- **Video dual ISO in 1080p**: the binned readout applies one gain (above).
- **Telling DRIVE / metering / AF-point apart**, or the light button from the rear dial: identical at the MPU (above).
- **ML audio capture** (`mlv_snd`): no ASIF stubs.
- `EngDrvOut`: an empty stub; Canon inlines it (MMIO store plus shadow store). Depth and frame timing had to go through Canon's own paths.
- **8-bit raw**: no packer code.
- **A left/right level meter** from `0xd8000190`: it is level and peak.
- **Hot-shoe flash eventprocs**: inert outside factory mode.
- **The SD clock**: Canon already runs its fastest divider.
- An early conclusion that the video path has no per-ISO table (from the ROM1 tune-data store) was **wrong** — see the sensor-config block above.

---

6. Open points

From a blind code review of the port and from testing:

- `FA_AudioRecStop` runs without checking that the recorder started (a full card or all 256 `Sample` names taken could assert).
- `CONFIG_MMU_EXTRA_PAGES` allocates with interrupts off (above).
- A REC press during the bit-depth zoom bounce can start a take mid-bounce.
- Peaking needs one ML menu open/close after power-on and after a depth change.
- Upstream's `PROP_LV_ACTION` whitelist entry for issue 307 and this port's mlv_lite fix are both in `main` since the merge (v132). They have not run together on the camera.
- **Crashes after crop-mode takes.** Five on v130, each 10 to 35 s after the take ended, all `LvJob.c:203` then `EvfState.c:660/584`. Changing many settings on v131 did not bring them back. Cause unknown.
- **Exposure steps in two Dual ISO takes** (`M06-1305`, `M06-1358`): about a third of a stop within one frame. Cause unknown.

- Firmware 1.2.0: 1.1.1 plus insertions (stubs move by a constant per ROM region); boots in QEMU, never run on a camera, and has no `ML-SETUP.FIR`.
- **Lossless:**
- 14-bit lossless at 59.94p (~80 MB/s at 2048 wide) needs a faster card than this one.
- 10-bit lossless would need DIGIC 5's 1/4 gain in front of a 12-bit encode.
- A failed encode ends the take, and an encode that never finishes leaves the encoder unused until a restart. Neither has happened on the camera since v111.
- **Gyro log:** see "Where the gyro stands" above.